

# Attention(어텐션)

정의

등장 배경

어텐션 함수(Attention Function)

Dot-Product Attention(닷-프로덕트 어텐션)

- 1) 어텐션 스코어(Attention Score) 구하기
- 2) 소프트맥스(softmax) 함수를 통해 어텐션 분포(Attention Distribution) 구하기
- 3) 각 인코더의 어텐션 가중치와 은닉 상태를 가중합하여 어텐션 값(Attention Value) 구하기
- 4) 어텐션 값과 디코더의  $t$  시점의 은닉 상태를 연결(Concatenate)
- 5) 출력층 연산의 입력이 되는  $\tilde{S}_t$ 를 계산
- 6)  $\tilde{S}_t$ 를 출력층의 입력으로 사용

다양한 종류의 어텐션(Attention)

결론

## 정의

- 신경망들의 성능을 높이기 위해 등장하여 이제는 AI 분야에서 대세 모듈로서 사용되고 있는 트랜스포머의 기반이 되는 중요한 메커니즘
- 어텐션의 기본 아이디어는 디코더에서 출력 단어를 예측하는 매 시점(time step)마다, 인코더에서의 전체 입력 문장을 다시 한 번 참고한다는 점
- 단, 전체 입력 문장을 전부 다 동일한 비율로 참고하는 것이 아니라, 해당 시점에서 예측해야 할 단어와 연관이 있는 입력 단어 부분을 좀 더 집중(attention)해서 보는 것이 핵심

## 등장 배경

- seq2seq 모델은 **인코더**에서 입력 시퀀스를 컨텍스트 벡터라는 하나의 고정된 크기의 벡터 표현으로 압축하고, **디코더**는 이 컨텍스트 벡터를 통해서 출력 시퀀스를 만드는 모델
- 이러한 RNN에 기반한 seq2seq 모델에는 크게 두 가지 문제가 있음
  - 첫째, 하나의 고정된 크기의 벡터에 모든 정보를 압축하려고 하니까 정보 손실이 발생함
  - 둘째, RNN의 고질적인 문제인 기울기 소실(vanishing gradient) 문제가 존재함
- 이로 인해 기계 번역 분야에서 입력 문장이 길면 번역 품질이 떨어지는 현상 발생

- 이를 해결하기 위해 입력 시퀀스가 길어지면 출력 시퀀스의 정확도가 떨어지는 것을 보정해주는 기법으로 어텐션(attention)이 등장함

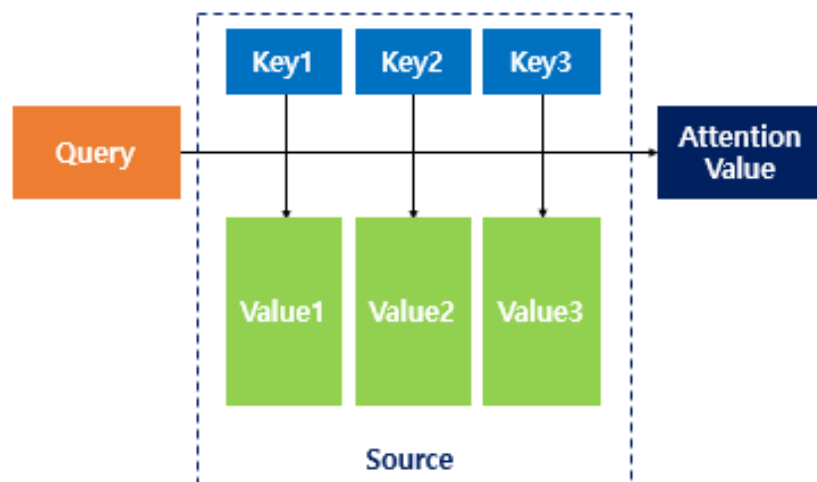
## 어텐션 함수(Attention Function)

- 이해를 돕기 위해 파이썬의 딕셔너리 자료형을 사용함
- 파이썬의 딕셔너리 자료형은 키(Key)와 값(Value)이라는 두 개의 쌍으로 구성되는데, 키를 통해서 매핑된 값을 찾아낼 수 있다는 특징이 있음

```
# 파이썬의 딕셔너리 자료형을 선언
# 키(Key) : 값(value)의 형식으로 키와 값의 쌍(Pair)을 선언한다.
dict = {"2017" : "Transformer", "2018" : "BERT"}

#2017이라는 키에 해당되는 값을 출력
print(dict["2017"])
>> Transformer

#2018이라는 키에 해당되는 값을 출력
print(dict["2018"])
>> BERT
```



[그림 1] 어텐션 함수의 개념

- 어텐션을 함수로 표현하면 주로 다음과 같이 표현

Attention(Q, K, V) = Attention Value

- 어텐션 함수는 주어진 쿼리(Query)에 대해서 모든 키(Key)와의 유사도를 각각 구함
- 구해낸 이 유사도를 키와 맵핑되어있는 각각의 값(Value)에 반영
- 유사도가 반영된 값(Value)을 모두 더해서 리턴하는데 이 값을 어텐션 값(Attention Value)이라고 함
- seq2seq + 어텐션 모델에서 Q, K, V에 해당되는 각각의 Query, Keys, Values는 각각 다음과 같음

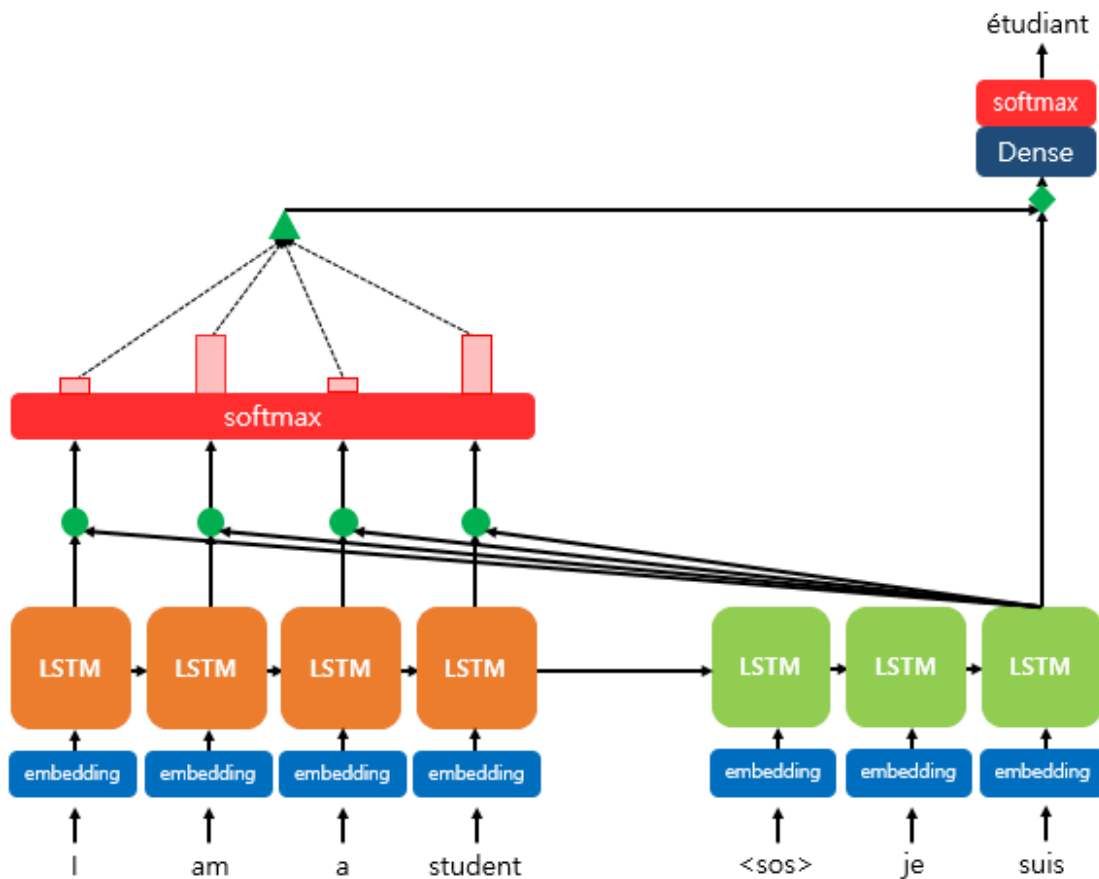
Q = Query : t 시점의 디코더 셀에서의 은닉 상태

K = Keys : 모든 시점의 인코더 셀의 은닉 상태들

V = Values : 모든 시점의 인코더 셀의 은닉 상태들

## Dot-Product Attention(닷-프로덕트 어텐션)

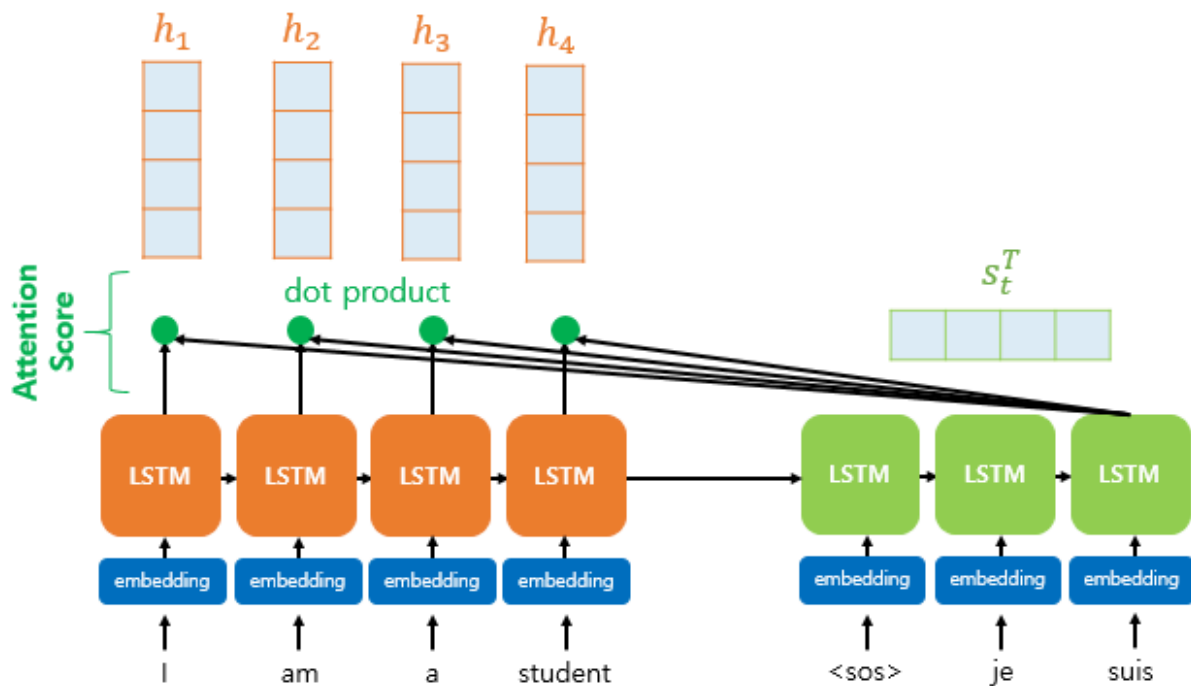
- 어텐션은 다양한 종류가 있는데 그 중에서도 수식적으로 가장 이해하기 쉬움
- 닷-프로덕트 어텐션과 다른 어텐션의 차이는 주로 중간 수식의 차이로 메커니즘 자체는 거의 유사함



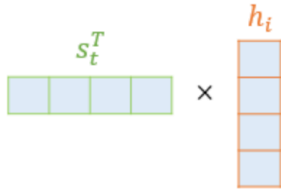
[그림 2] 어텐션 메커니즘

- 디코더의 세번째 LSTM 셀에서 출력 단어를 예측할 때, 어텐션 메커니즘을 사용하는 케이스
- 디코더의 첫번째, 두번째 LSTM 셀은 이미 어텐션 메커니즘을 통해 je와 suis를 예측하는 과정을 거쳤다고 가정
- 디코더의 세번째 LSTM 셀은 출력 단어를 예측하기 위해서 인코더의 모든 입력 단어들의 정보를 다시 한번 참고함
- 소프트맥스 함수를 통해 나온 결과값은 I, am, a, student 단어 각각이 출력 단어를 예측할 때 얼마나 도움이 되는지의 정도를 수치화한 값(빨간 직사각형)
- 각 입력 단어가 디코더의 예측에 도움이 되는 정도가 수치화하여 측정되면 이를 하나의 정보로 담아서 디코더로 전송함(초록색 삼각형)
- 이로 인해 디코더는 출력 단어를 더 정확하게 예측할 확률이 높아짐

## 1) 어텐션 스코어(Attention Score) 구하기



- $h_1, h_2, \dots, h_n$  : 인코더의 시점(time step)을 각각 1, 2, ... N이라고 하였을 때 인코더의 은닉 상태(hidden state)
- $S_t$  : 디코더의 현재 시점(time step)  $t$ 에서의 디코더의 은닉 상태(hidden state)
- $h_n, S_t$  : 동일한 4차원
- $a_t$  :  $t$ 번째 단어를 예측하기 위한 어텐션 값(Attention Value)
  - 이전에 배운 Seq2Seq에서 시점  $t$ 에서 출력 단어를 예측하기 위해서 디코더의 셀은 두 개의 입력값을 필요로 하는데, 바로 이전 시점인  $t-1$ 의 은닉 상태와 이전 시점  $t-1$ 에 나온 출력 단어였음
- 어텐션 스코어란 현재 디코더의 시점  $t$ 에서 단어를 예측하기 위해, 인코더의 모든 은닉 상태 각각이 디코더의 현재 시점의 은닉 상태  $S_t$ 와 얼마나 유사한지를 판단하는 스코어값
- 닷-프로덕트 어텐션에서는 이 스코어 값을 구하기 위해  $S_t$ 를 전치(transpose)하고 각 은닉 상태와 내적(dot product)을 수행하고 그 결과 모든 어텐션 스코어 값은 스칼라임



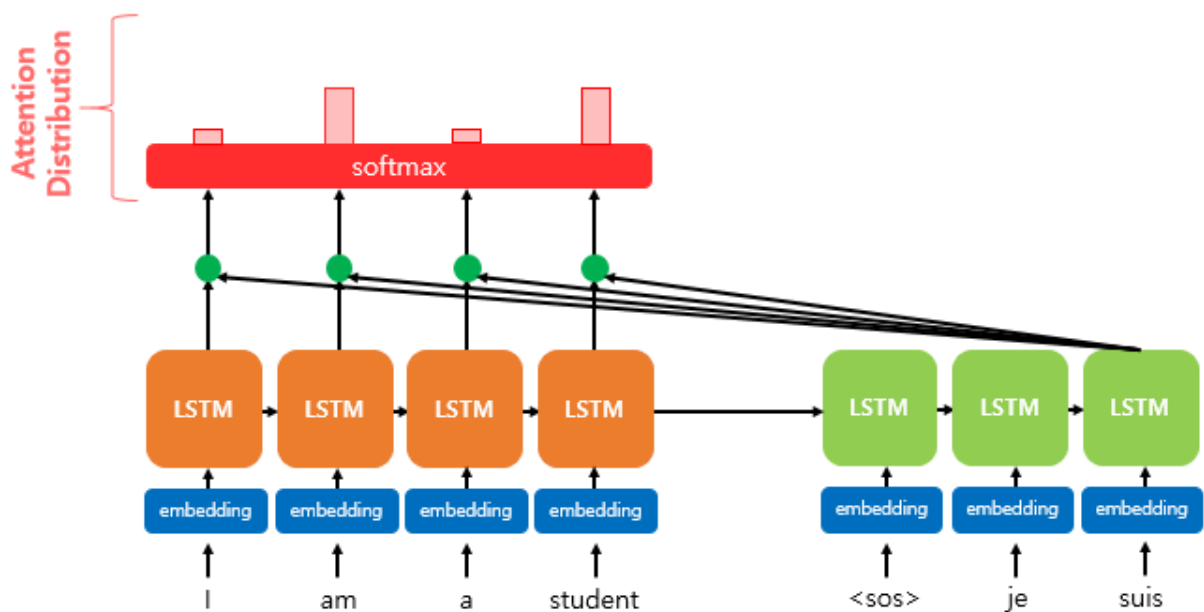
어텐션 스코어 함수를 정의해보면 다음과 같습니다.

$$\text{score}(s_t, h_i) = s_t^T h_i$$

$s_t$ 와 인코더의 모든 은닉 상태의 어텐션 스코어의 모음값을  $e^t$ 라고 정의하겠습니다.  $e^t$ 의 수식은 다음과 같습니다.

$$e^t = [s_t^T h_1, \dots, s_t^T h_N]$$

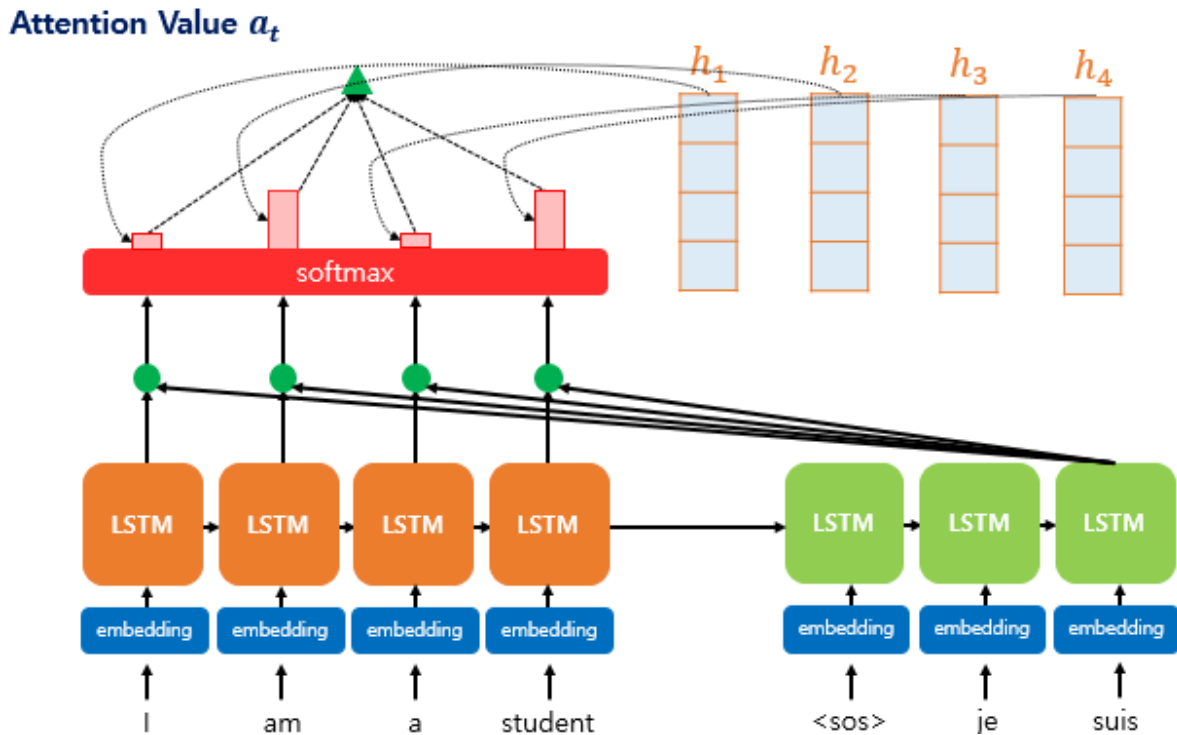
## 2) 소프트맥스(softmax) 함수를 통해 어텐션 분포(Attention Distribution) 구하기



- 어텐션 분포(Attention Distribution) :  $e^t$ 에 소프트맥스 함수를 적용하여, 모든 값을 합하면 1이 되는 확률 분포
- 각각의 값은 어텐션 가중치(Attention Weight)라고 함
- 소프트맥스 함수를 적용하여 얻은 출력값인 I, am, a, student의 어텐션 가중치를 각각 0.1, 0.4, 0.1, 0.4라고 하면 이들의 합은 1
- 디코더의 시점 t에서의 어텐션 가중치의 모음값인 어텐션 분포  $a^t$  을 식으로 정의하면 다음과 같음

$$\alpha^t = \text{softmax}(e^t)$$

### 3) 각 인코더의 어텐션 가중치와 은닉 상태를 가중합하여 어텐션 값(Attention Value) 구하기



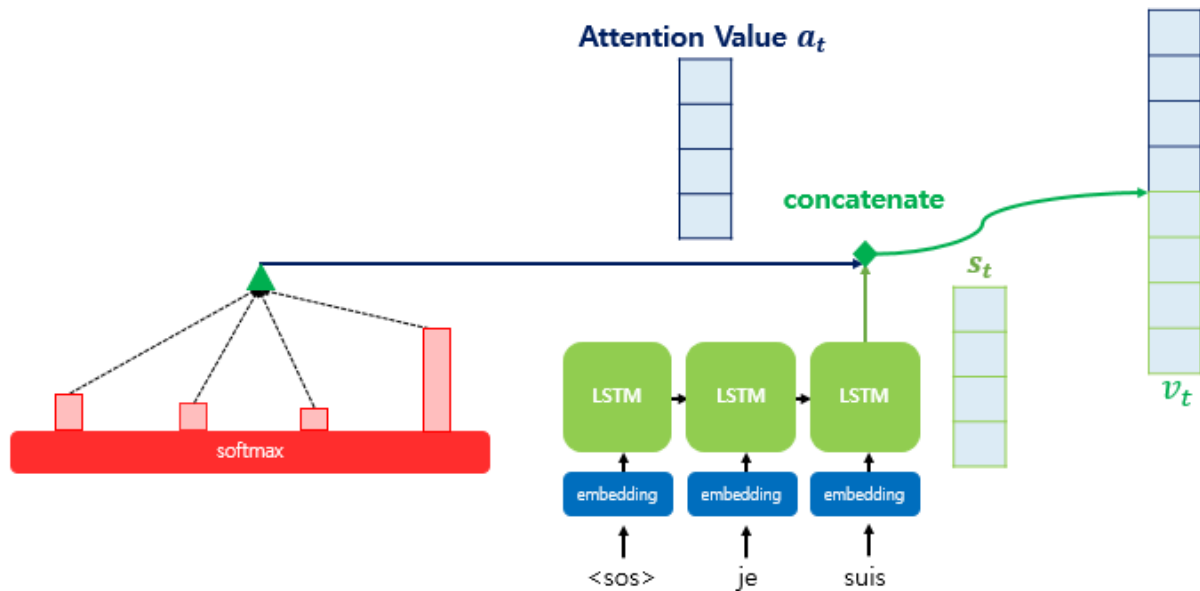
- 지금까지 준비해온 정보들을 하나로 합치는 단계
- 어텐션의 최종 결과값을 얻기 위해서 각 인코더의 은닉 상태와 어텐션 가중치값들을 곱하고, 최종적으로 모두 더해 가중합(Weighted Sum)을 진행함
- 어텐션의 최종 결과. 즉, 어텐션 함수의 출력값인 어텐션 값(Attention Value)  $\alpha_t$ 에 대한 식은 다음과 같음

$$a_t = \sum_{i=1}^N \alpha_i^t h_i$$

- 어텐션 값  $a_t$ 는 종종 인코더의 문맥을 포함하고 있다고하여, 컨텍스트 벡터(context vector)라고도 함

- seq2seq에서는 인코더의 마지막 은닉 상태를 컨텍스트 벡터라고 부르는 것과 비교

#### 4) 어텐션 값과 디코더의 t 시점의 은닉 상태를 연결 (Concatenate)



- $a_t$  : 어텐션 함수의 최종값인 어텐션 값
- $v_t$  : 어텐션 값(Attention Value)  $a_t$ 를  $s_t$ 와 결합(concatenate)한 하나의 벡터로서 어텐션 메커니즘의 핵심
- $v_t$ 를  $\hat{y}$  예측 연산의 입력으로 사용하므로써 인코더로부터 얻은 정보를 활용하여  $\hat{y}$ 를 예측함

#### 5) 출력층 연산의 입력이 되는 $\tilde{s}_t$ 를 계산

$$\tanh \left( W_c \times v_t \right) = \tilde{s}_t$$



- 논문에서는  $v_t$ 를 바로 출력층으로 보내기 전에 신경망 연산을 한 번 더 추가함
- 가중치 행렬과 곱한 후에 하이퍼볼릭탄젠트 함수를 지나도록 하여 출력층 연산을 위한 새로운 벡터인  $\tilde{s}_t$ 를 얻음. 즉, 출력층의 t시점에서의 입력이,
  - seq2seq :  $s_t$
  - 어텐션 메커니즘 :  $\tilde{s}_t$
- 식으로 표현하면 다음과 같음
  - $W_c$  : 학습 가능한 가중치 행렬
  - $b_c$  : 편향(bias) (그림에서 편향은 생략함)

## 6) $\tilde{s}_t$ 를 출력층의 입력으로 사용

$\tilde{s}_t$ 를 출력층의 입력으로 사용하여 예측 벡터를 얻음

$$\hat{y}_t = \text{Softmax}(W_y \tilde{s}_t + b_y)$$

## 다양한 종류의 어텐션(Attention)

| 이름                     | 스코어 함수                                                                                              | Defined by             |
|------------------------|-----------------------------------------------------------------------------------------------------|------------------------|
| <i>dot</i>             | $score(s_t, h_i) = s_t^T h_i$                                                                       | Luong et al. (2015)    |
| <i>scaled dot</i>      | $score(s_t, h_i) = \frac{s_t^T h_i}{\sqrt{n}}$                                                      | Vaswani et al. (2017)  |
| <i>general</i>         | $score(s_t, h_i) = s_t^T W_a h_i$ // 단, $W_a$ 는 학습 가능한 가중치 행렬                                       | Luong et al. (2015)    |
| <i>concat</i>          | $score(s_t, h_i) = W_a^T \tanh(W_b[s_t; h_i])$ , $score(s_t, h_i) = W_a^T \tanh(W_b s_t + W_c h_i)$ | Bahdanau et al. (2015) |
| <i>location - base</i> | $\alpha_t = \text{softmax}(W_a s_t)$ // $\alpha_t$ 산출 시에 $s_t$ 만 사용하는 방법.                           | Luong et al. (2015)    |

## 결론

---

- 어텐션은 처음에는 RNN 기반의 seq2seq의 성능을 보정하기 위한 목적으로 등장함
- 현재에 이르러서는 어텐션 스스로가 기존의 seq2seq를 대체하는 방법이 되어가고 있음
- 트랜스포머(Transformer)에서 활용됨